



西南科技大学

Southwest University of Science and Technology

信息隐藏技术论文

题目名称： 文法类隐写与分析技术的实现

学 院 名 称 计 算 机 科 学 与 技 术 学 院

专 业 名 称

学 生 姓 名

学 号

指 导 教 师 周 宏 毅

二〇二四年十月

文法类隐写与分析技术的实现

摘要：随着数字通信的迅速发展，信息安全成为了一个重要议题。隐写术，作为一种信息隐藏技术，允许用户在不引起注意的情况下传输敏感数据。本文通过实现基于文法类的隐写与分析技术，旨在提高数据隐藏的隐蔽性和鲁棒性。我们首先分析了常见的文件格式，包括图像、音频和视频文件，以确定最适合数据嵌入的冗余区域。然后，我们设计了一种算法，该算法能够在保持文件格式完整性的同时，将信息嵌入到这些区域中。此外，我们还设计相应算法，用于检测和提取隐藏的数据。在实验部分，我们对不同文件格式进行了测试，包括 PNG 文件、BMP 文件和 BIN 文件。最后，我们讨论了该技术在实际应用中的潜在价值，包括但不限于数字版权管理、安全通信和个人隐私保护。

关键词：隐写术；文法类隐写；数据隐藏；信息安全；文件格式分析

Implementation of Grammar-Based Steganography and Analysis Techniques

Abstract: With the rapid development of digital communication, information security has become an important issue. Steganography, as a technique for information hiding, allows users to transmit sensitive data without drawing attention. This paper aims to improve the concealment and robustness of data hiding by implementing grammar-based steganography and analysis techniques. We first analyze common file formats, including image, audio, and video files, to identify the most suitable redundant areas for data embedding. Subsequently, we design an algorithm capable of embedding information into these areas while maintaining the integrity of the file format. Additionally, we develop corresponding algorithms for detecting and extracting hidden data. In the experimental section, we test different file formats, including PNG, BMP, and BIN files. Finally, we discuss the potential value of this technology in practical applications, including but not limited to digital rights management, secure communication, and personal privacy protection.

Key words: Steganography; Grammar-based steganography; Data hiding; Information security; File format analysis

目 录

任务一：完成对头文件冗余的数据隐写、分析和提取	1
1.1 序言	1
1.2 选择实验对象.....	1
1.2.1 PNG 文件格式	1
1.2.2 BMP 文件格式	1
1.3 算法设计	2
1.4 核心代码	2
1.5 实验结果	4
任务二：完成文件尾部追加的数据隐写、分析和提取	6
2.1 序言	6
2.2 文件格式	6
2.3 算法设计	6
2.4 关键代码	6
2.5 实验结果	7
任务三：完成位图格式图像的数据区进行数据直接覆盖方式的隐写、分析和提取。	9
3.1 序言	9
3.2 文件格式	9
3.3 算法设计	9
3.4 核心代码	10
3.5 实验结果	11
结论	15
参考文献	16
附录	17

任务一：完成对头文件冗余的数据隐写、分析和提取

1.1 序言

这种技术涉及对文件头信息的冗余部分进行数据隐藏。这通常需要对文件格式有深入的了解，以便找到可以嵌入数据而不引起注意的部分。分析和提取过程则需要检测这些异常模式并从中恢复信息。

1.2 选择实验对象

1.2.1 PNG 文件格式

PNG 文件由文件签名、数据块组成。

- 文件签名 (sig): PNG 文件以一个 8 字节的固定签名 (89 50 4E 47 0D 0A 1A 0A) 开始, 标识文件是 PNG 格式。
- 数据块 (Chunks), 包括但不限于:
 - IHDR: 图像头信息, 包含图像的宽度、高度、颜色类型等。
 - IDAT: 图像数据。
 - IEND: 文件结束标志

在 PNG 文件中, 最适合数据嵌入的区域通常是 IDAT 块, 因为它包含了图像的压缩数据。即使在 IDAT 块中嵌入数据, 只要保持 CRC 校验码的正确性, 就不会影响 PNG 文件的解析。由于 PNG 文件格式有严格的头文件规范, 直接在头部插入数据会导致文件无法正常解析, 因此无法作为本实验实验对象。

1.2.2 BMP 文件格式

BMP 文件头部信息通常由以下部分组成:

文件头 (14 字节): 签名 (2 字节)、文件大小 (4 字节)、保留 (4 字节) 和偏移量 (4 字节)

DIB 头 (位图信息头, 通常为 40 字节): 标头大小 (4 字节)、宽度 (4 字节)、高度 (4 字节)、平面 (2 字节)、位数 (2 字节)、压缩 (4 字节)、图像大小 (4 字节)、每米 X 像素 (4 字节)、Y 像素每米 (4 字节)、调色板中的颜色 (4 字节)、重要颜色计数 (4 字节)

这意味着标准 BMP 文件的头部（前 54 字节）保存文件信息，可以利用一些冗余字节嵌入隐藏数据，不会影响 .bmp 文件的显示。因此可以选择 BMP 文件作为实验对象。

1.3 算法设计

在 Python 中，有一个官方模块 struct，该模块提供了内置的 pack 和 unpack 函数，用于高效地处理二进制数据。我们可以利用这两个函数对 BMP 文件的文件头和信息头进行解析，以确保能够准确读取到文件的各个关键字段。

例如，文件头中的前两个字节表示文件类型，我们需要确保它们为'BM'，以确认该文件确实是一个有效的 BMP 文件。接着，我们可以提取位图大小、像素数据偏移量、图像宽度、高度以及色彩平面数等信息。

数据隐写：通过 hide_data_in_bmp_reserved 函数，将用户输入的数据隐写到 BMP 文件的保留字段中，保留字的主要用途是在特定情况下保留空间，在大多数情况下，这些字段并不影响图像显示，因此是一个理想的隐写位置。保留字段的大小固定为 4 个字节，因此，如果用户输入的数据超过 4 个字符，将进行截断；如果不足 4 个字符，则用\0 填充。

数据提取：通过 extract_data_from_bmp_reserved 函数，从指定的 BMP 文件中读取保留字段，提取隐藏的信息。读取后将数据进行解码，并去除填充字符\0。

数据分析：通过查看保留字是否为 0 判断是否隐写。

1.4 核心代码

```
# 读取并格式化输出 BMP 文件头信息
def bmp_info(file_path):
    try:
        with open(file_path, 'rb') as f:
            bmp_data = f.read(54) # 读取前 54 个字节以获取头部信息
            if len(bmp_data) < 54:
                print("文件太短，无法作为有效的 BMP 文件。")
                return None
            # 解压 BMP 头部
            unpackbuf = struct.unpack('<ccIIIIIIHHIIIIIII', bmp_data)
            # 检查是否是 BMP 文件
```

```

        if unpackbuf[0] != b'B' or unpackbuf[1] != b'M':
            print("不是有效的 BMP 文件。")
            return None
        print("BMP 文件信息: ")
        for key, value in bmp_info_result.items():
            print(f"{key}: {value}")
        return bmp_info_result
    except Exception as e:
        print(f"发生错误: {e}")
        return None

# 数据隐写: 将数据隐藏在 BMP 文件头保留字段
def hide_data_in_bmp_reserved(file_path, output_path, data_to_hide):
    data_to_hide = data_to_hide[:4].ljust(4, '\0') # 保留字 4 字节
    data_bytes = data_to_hide.encode('utf-8')
    shutil.copy(file_path, output_path)
    with open(output_path, 'r+b') as file:
        file.seek(6) # 保留字段偏移位置
        file.write(data_bytes)
    print("-----信息已成功隐写到 BMP 文件头保留字段中。-----")

# 提取: 从 BMP 文件头保留字段中读取隐藏的数据
def extract_data_from_bmp_reserved(file_path):
    with open(file_path, 'rb') as file:
        file.seek(6) # 定位到保留字段的偏移位置
        hidden_data = file.read(4) # 读取 4 字节的隐藏数据
        extracted_data = hidden_data.decode('utf-8').strip('\0')
        print(f"提取的隐藏信息: {extracted_data}")
        return extracted_data

# 分析: 检测 BMP 文件头, 确认隐写内容
def analyze_bmp_header(file_path, expected_hidden_data):
    bmp_header = bmp_info(file_path)

```

```

if not bmp_header:
    print("-----分析失败，文件头信息读取错误。-----")
    return
hidden_data = extract_data_from_bmp_reserved(file_path)
if hidden_data == expected_hidden_data:
    print("-----分析结果：隐藏信息符合预期，数据完整。-----")
else:
    print(f"-----分析结果：隐藏信息不匹配，提取到的数据为
'{hidden_data}', 与预期不符。-----")
    print("-----分析完成。-----")

```

1.5 实验结果

页	hide_header_python.bmp x	python.bmp
0	42 4D 0A 03 00 00 48 49 44 45 76 00 00 00 28 00	BM . . . H I D E v . . . (.
1	00 00 21 00 00 00 21 00 00 00 01 00 04 00 00 00	. . ! . . . !
2	00 00 94 02 00 00 00 00 00 00 00 00 00 00 00 00	
3	00 00 00 00 00 00 00 00 00 00 00 00 80 00 00 80	
4	00 00 00 80 80 00 80 00 00 00 80 00 80 00 80 80	

图 1-1 隐写后文件保留字



图 1-2 左图为原图，右图为隐写文件头后的图片

进行隐写后文件正常显示，且无差别，但图片信息有隐写数据。隐写成功。



图 1-3 BMP 文件查看

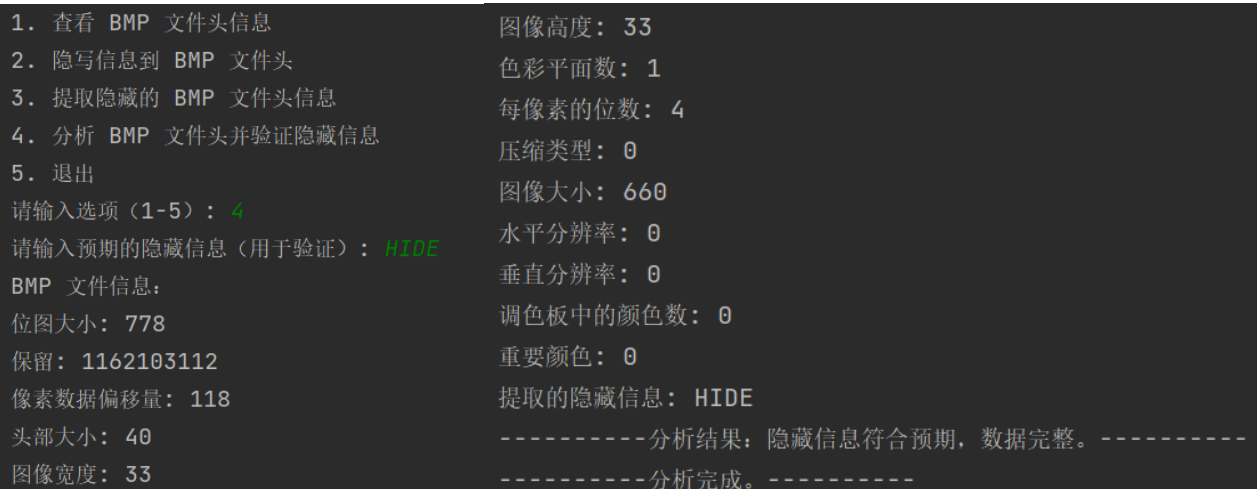
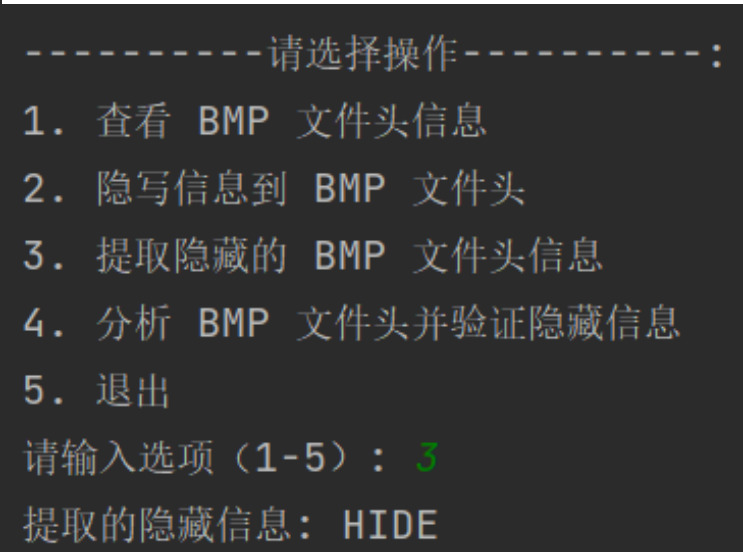
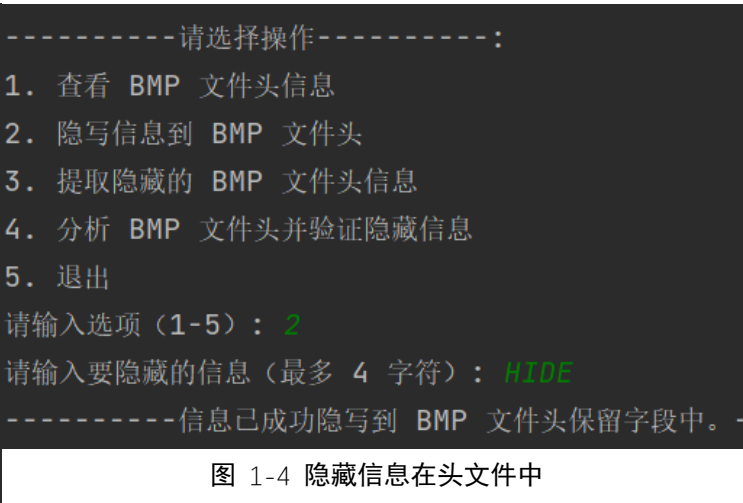


图 1-6 分析 BMP 头文件

任务二：完成文件尾部追加的数据隐写、分析和提取

2.1 序言

在数字隐写术中，利用文件的尾部进行数据隐写是一种有效的方法。我们可以将数据以隐蔽的方式追加到 BMP 文件的末尾，而不会影响图像的正常显示。

2.2 文件格式

BMP 文件具有固定的头部结构和数据区，但其尾部并没有严格的格式定义。文件尾部的数据不会被 BMP 图像渲染引擎读取，这使得我们可以在文件尾部自由地追加数据，而不影响图片的完整性。

2.3 算法设计

数据隐写：将原 BMP 文件复制到一个新的输出文件，以追加模式打开输出文件，并将用户输入的数据编码为字节后写入文件的末尾。

数据提取：在提取前先得读取文件头中的 BITMAPFILEHEADE 数值，然后使用 file.seek 方法移动到 BITMAPFILEHEADE 后开始读取隐藏的数据。读取的字节将被解码为字符串并打印出来，以供用户查看。

2.4 关键代码

```
# 隐写：将数据追加到 BMP 文件尾部
def hide_data_in_bmp_footer(file_path, output_path, data_to_hide):
    # 确保输出文件存在
    shutil.copy(file_path, output_path)
    with open(output_path, 'ab') as file: # 以追加模式打开文件
        data_bytes = data_to_hide.encode('utf-8') # 将数据编码为字节
        file.write(data_bytes)
    print("-----信息已成功隐写到 BMP 文件尾部。-----")

# 提取：从 BMP 文件尾部读取隐藏的数据
def detect_hidden_data_in_bmp_footer(file_path):
    with open(file_path, 'rb') as file:
        # 读取文件头中的文件长度信息 (3-6 字节)
        file.seek(2)
        file_length_in_header = struct.unpack('<I', file.read(4))[0]
        actual_file_length = os.path.getsize(file_path)
```

```

if actual_file_length > file_length_in_header:
    print(
        f"检测到隐写信息: 文件实际长度为 {actual_file_length}, 文件头中保存的
长度为 {file_length_in_header}。")
    start_pos = file_length_in_header
    file.seek(start_pos, os.SEEK_SET)
    # 读取隐藏信息, 长度为实际文件长度减去文件头中记录的长度
    hidden_data = file.read(file_length_in_header)
    print(hidden_data)
else:
    print("-----未检测到隐写信息。-----")
    return False, ""

```

2.5 实验结果



图 2-1 左图 of 小数据隐写, 右图 of 大数据隐写

都显示正常, 且都能通过 010 查看到隐藏信息.

python.bmp																hide_large_footer_python.bmp																hide_small_footer_python.bmp x																					
0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F						
00	00	FF	FF	FF	03	30	33	30	FF	FF	FF	FF	FF	FF	FF	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...				
FF	FF	F0	00	00	00	FF	FF	FF	F0	0F	00	0F	FF	FF	FF	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...			
FF	FF	FF	FF	FF	FF	F0	00	00	00	6E	69	68	61	6F	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...

图 2-2 小数据 010 查看

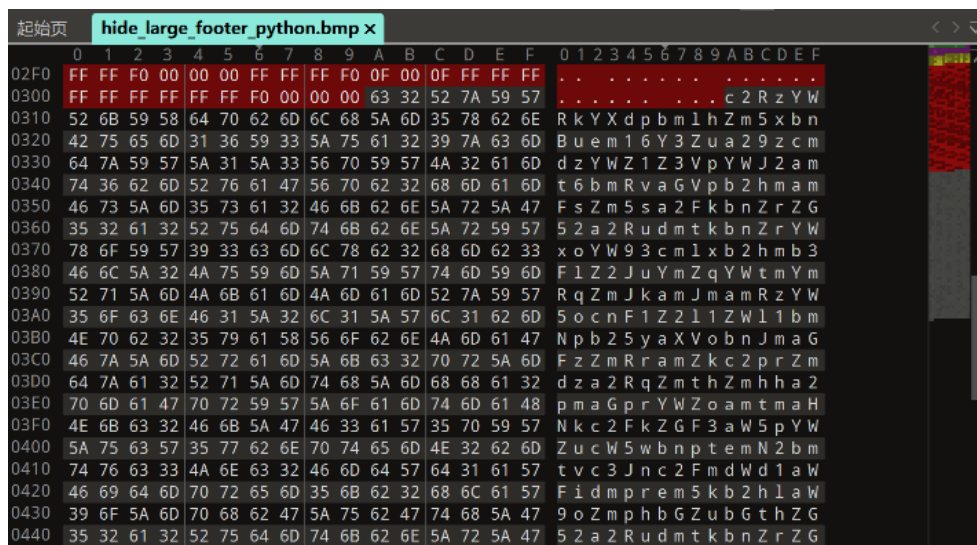


图 2-3 大数据 010 查看

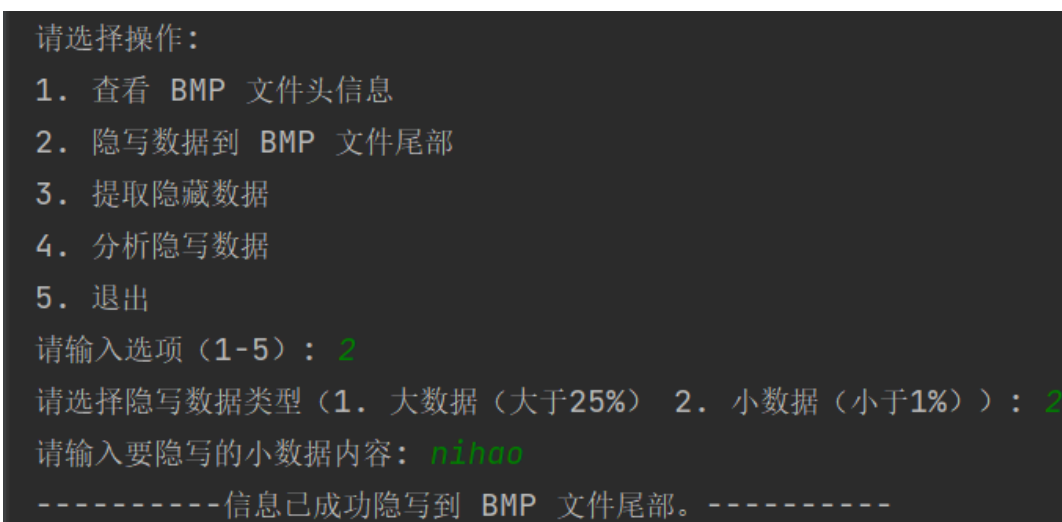


图 2-4 隐写数据到尾部



任务三：完成位图格式图像的数据区进行数据直接覆盖方式的隐写、分析和提取。

3.1 序言

在数字图像处理与隐写术领域，BMP（位图）文件因其结构简单、无压缩特性而成为隐写的理想载体。通过将隐秘数据嵌入图像的数据区，可以实现信息隐藏，保护数据内容的安全和隐私。本文讨论一种直接覆盖方式的隐写方法，包括对顺序与离散隐写的实现与分析方法，使得信息嵌入过程既稳定又难以察觉。

3.2 文件格式

BMP 格式文件数据区主要存储像素数据，以 RGB 格式呈现。无压缩的 BMP 图像数据区结构清晰，是直接数据覆盖的理想区域。

在数据直接覆盖隐写中，像素数据区用于存储隐秘信息，数据直接覆盖隐写不会改变文件头和信息头，因此不会影响图像结构完整性和显示效果。

3.2.1 BMP 文件头与调色板

在读取 BMP 文件时，首先需要理解 BMP 文件的结构，包括文件头、信息头和调色板。以下是 BMP 文件头的重要组成部分：

- 文件头（14 字节）：包括文件标识符（'BM'），文件大小，保留字，和数据偏移（bfoffbits）。
- 信息头（通常为 40 字节）：包括图像宽度、高度、颜色平面数、每像素位数、压缩类型等信息。
- 调色板：对于 8 位及以下的 BMP 文件，调色板用于存储颜色信息。

3.3 算法设计

在隐写过程中，需要获取 bfoffbits（数据偏移），以确定像素数据的起始位置。该值是一个 DWORD（4 字节），通常在文件头的第 10 个字节处读取。

数据隐写：

- 顺序隐写：将要隐写的数据转换为字节序列，并将其字节长度添加在序列前，用于提取数据时确定其长度，将整个数据序列从像素数据区开头直接写入，适

用于连续数据的隐藏。

- 离散隐写：离散隐写通过随机分布位置将数据字节嵌入数据区，以增强数据隐藏的隐秘性。隐写时随机选择不重复的位置存储数据，并保存一个位置索引文件，以便于后续提取。由于离散性，提取过程中依赖位置索引文件，确保数据的正确复原。

数据提取：

- 顺序隐写数据提取：从图像数据区的前 4 字节读取数据长度，根据长度从数据区顺序提取隐藏的数据字节，并转码为原始信息。此过程无需外部索引文件，确保提取的简便性。
- 离散隐写数据提取：读取位置索引文件，从记录的随机位置顺序提取数据字节，并解码复原出隐藏信息。

数据分析：

- 顺序隐写数据分析：通过判断数据区开头的 4 字节内容是否为合理的数据长度，以判断是否存在顺序隐写数据。如果前 4 字节的整数值过大或超出合理范围，则判定文件无隐写数据。
- 离散隐写数据分析：通过验证位置索引文件的存在和内容的合理性，确定是否进行离散隐写，结合索引位置进行数据隐写验证。

3.4 核心代码

```
def read_bmp_header(file_path):
    with open(file_path, 'rb') as f:
        f.seek(10)
        bfoffbits = struct.unpack('<I', f.read(4))[0]
        return bfoffbits + 0x54

# 隐写：顺序数据覆盖
def hide_data_sequential(bmp_file, data, output_file):
    start_position = read_bmp_header(bmp_file)
    print(f"数据隐写起始位置: {start_position}") # 打印起始位置
    with open(bmp_file, 'rb') as f:
        bmp_data = f.read()

    if len(data) > len(bmp_data) - start_position:
        raise ValueError("隐藏的数据过大，无法放入 BMP 文件中")
```

```

    hidden_data = bmp_data[:start_position] + data.encode() +
    bmp_data[start_position + len(data):]

    with open(output_file, 'wb') as f:
        f.write(hidden_data)

# 隐写：离散数据覆盖（使用索引追踪）
def hide_data_discrete(bmp_file, data, output_file, index_file):
    start_position = read_bmp_header(bmp_file)
    print(f"数据隐写起始位置: {start_position}") # 打印起始位置
    with open(bmp_file, 'rb') as f:
        bmp_data = bytearray(f.read())

    # 修改后的检查条件
    if len(data) > len(bmp_data) - start_position:
        raise ValueError("隐藏的数据过大，无法放入 BMP 文件中")

    data_bytes = data.encode()
    indices = []

    for i in range(len(data_bytes)):
        position = start_position + random.randint(0, (len(bmp_data) -
start_position - 1) // 10) * 10
        bmp_data[position] = data_bytes[i]
        indices.append(position)

    with open(output_file, 'wb') as f:
        f.write(bmp_data)

    with open(index_file, 'w') as f:
        json.dump(indices, f)

# 提取：从 BMP 文件中读取顺序数据（自动提取长度）
def extract_data_sequential(bmp_file, length):
    start_position = read_bmp_header(bmp_file)
    with open(bmp_file, 'rb') as f:
        f.seek(start_position)
        hidden_data = f.read(length)
    return hidden_data.decode(errors='ignore')

# 提取：从 BMP 文件中读取离散数据
def extract_data_discrete(bmp_file, index_file):
    with open(index_file, 'r') as f:

```



```
indices = json.load(f)

hidden_data = bytearray()

with open(bmp_file, 'rb') as f:
    for index in indices:
        f.seek(index)
        hidden_data.append(f.read(1)[0])

return hidden_data.decode(errors='ignore')
```

3.5 实验结果

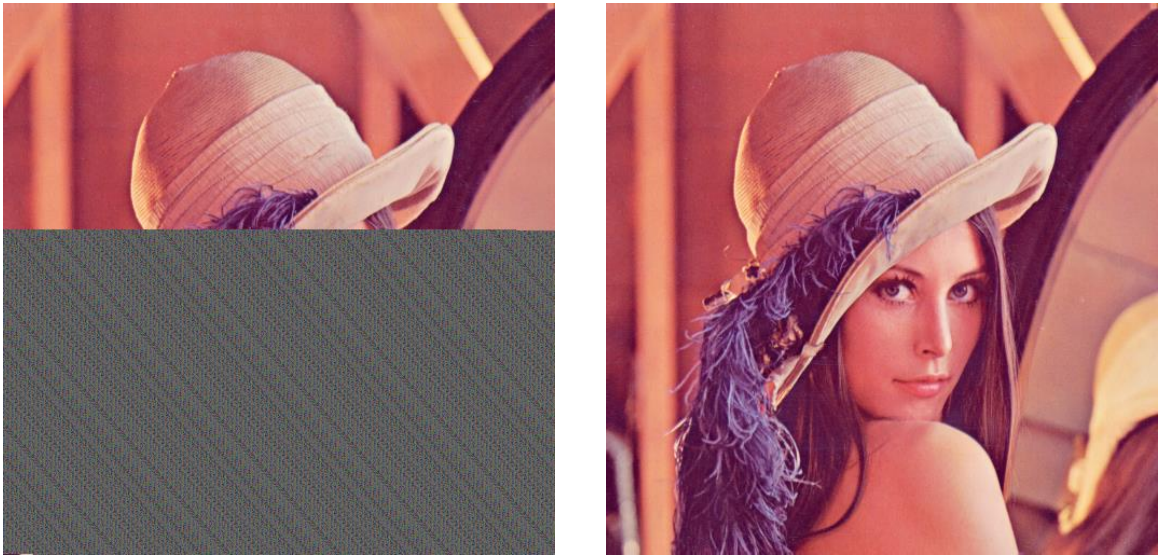


图 3-1 左图为顺序隐写后图片，右图为原图

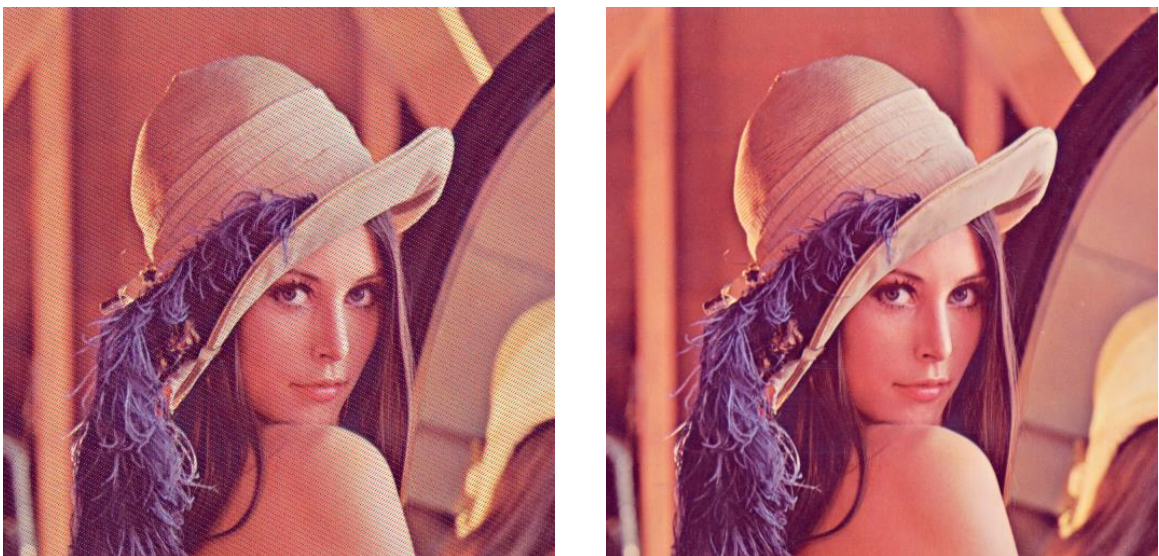


图 3-2 左图为离散隐写后图片，右图为原图

可以看出顺序隐写后会改变图片大面积色块，而离散隐写后只会改变少量像素点。

这是因为顺序隐写会连续覆盖像素值，尤其是当隐藏的数据量较大时，覆盖的区域会影响到相邻多个像素，可能形成明显的色块或区域变化。

而离散隐写只覆盖少量随机选择的像素点，每次隐写数据只会改变几处像素的颜色，导致整体视觉效果变化不明显。这样的隐写方式不会形成大面积的色块，因此相对更难被发现。

```
选择操作：
1. 隐写顺序数据
2. 隐写离散数据
3. 提取顺序隐藏数据
4. 提取离散隐藏数据
5. 退出
请输入你的选择 (1-5): 1
数据隐写起始位置: 138
顺序隐写完成，生成文件: ./hide_sequential.bmp
```

图 3-3 顺序隐写操作

离散隐写会生成一个 index_map.json 索引文件，文件内容为：

```
index_map.json  +  X
架构: <未选择架构>
1 [53, 496, 513, 154, 329, 111, 611, 281, 583, 185, 152, 637, 194, 273, 338, 707, 593]
```

图 3-4 索引文件内容

```
选择操作：
1. 隐写顺序数据
2. 隐写离散数据
3. 提取顺序隐藏数据
4. 提取离散隐藏数据
5. 退出
请输入你的选择 (1-5): 2
数据隐写起始位置: 138
离散隐写完成，生成文件: ./hide_discrete.bmp，索引已保存至 ./index.json
```

图 3-4 离散隐写操作

结论

实验完成了对不同文件类型的隐写、分析和提取，主要结论如下：

文件头隐写：通过直接在文件头部隐写数据，可以有效嵌入小量隐私信息，且简单易于实现。

文件尾部隐写：对于大数据量的隐写，可以直接追加数据到文件尾部，适合需要隐藏较大信息的场景。小数据隐写同样有效。

BMP 图像隐写：在 BMP 文件数据区通过顺序和离散隐写实现了信息的嵌入，确保了数据隐写的隐蔽性与有效性。

分析与提取功能：数据分析和提取功能的实现增强了隐写应用的便捷性，使得用户可以轻松判断文件中是否存在隐写数据，并恢复原始信息。

参考文献

- [1] Martin, G. Control of electronic resources in Australia. In Pattle, L. W., & Cox, B. J. (Eds.), Electronic resources: selection and bibliographic control. New York: The Haworth Press, 1966: 85-96.
- [2] Wang, W. L. Preparation and performance study of fluorescent carbon nanomaterials [D]. Shanghai: Shanghai Normal University, 2019.
- [3] Westfeld, A. (2001). F5 — A steganographic algorithm: High capacity despite better detection. In International Workshop on Information Hiding, 2001, 289-302.
- [4] Zhang, Y., & Wang, X. (2018). A survey of image steganography techniques. International Journal of Computer Applications, 182(30), 30-37.
- [5] C. H. S. (2014). Practical approaches to hiding information in digital images. IEEE Transactions on Information Forensics and Security, 9(1), 11-25.
- [6] Petitcolas, F. A. P., Anderson, R., & Kuhn, M. G. (1999). Information hiding—A survey. Proceedings of the IEEE, 87(7), 1062-1078.

附录